# Advanced Data Management: chapters 1–6 on two pages

blue = topic, red ! = the mistake that costs points. $B$ pages of a table, $N$ or $|R|$ rows, $r$ rows per page, $M$ buffer pages (`work_mem`/8 KB: 4 MB = 512), $h$ tree height, $F$ fanout, $s$ selectivity, $d$ distinct values. `adm_lab`: `orders` 500,000 rows / 3,185 p / 157 per p; `customers` 10,000 / 73. `shopstream`: `events` 50M / 1M p / 50 per p, `users` 2M / 30,000 p.

## 1 Architecture and the life of a query

**Five subsystems**: connection manager (sessions, one worker per connection) ▹ query processor (parser, optimiser = *planner*, executor) ▹ storage manager (pages/files, access methods = indexes, *buffer pool*) ▹ transaction manager (locks, log, recovery) ▹ catalog (schema + statistics: `pg_class`, `pg_stats`). **Symptom → owner**: Seq Scan where an index is faster: planner. Lost update: transaction mgr (locks). Low hit ratio: buffer pool. "column does not exist": binder via catalog. Estimate 1,000× off: stale statistics, `ANALYZE`. **Five stages**: parse (grammar only) → bind (names, types, permissions, *against the catalog*) → rewrite (relational algebra; predicate pushdown = logical) → optimise (access path, join algorithm, join order, by cost from statistics = physical) → execute. **Why an own buffer pool, not the OS cache**: knows the access pattern (scan = touch once; OS LRU floods), the write order (log before data page; OS writes in any order), pinning + memory budget (double buffering). **Iterator model**: `open/next/close`; requests down, tuples up, pipelined. **Streaming**: scans, filter, projection, probe side, Nested Loop, Merge Join, GroupAggregate, Limit. **Blocking** (breaker): Sort, Hash (build side), HashAggregate. Read EXPLAIN bottom-up; under `Hash Join` the child under `Hash` is the build side. **SQL runs** FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY: alias only in ORDER BY; aggregate only in HAVING; selected columns grouped or aggregated. **OLTP**: short transactions, whole rows: row store. **OLAP**: scans of few columns: column store ($1/k$ of the pages for 1 of $k$ columns, compresses; but 1 row update = $k$ pages).

! "The parser catches a misspelled column" (binder does). "The executor picks the scan" (planner does). "Hash Join is blocking" (only its build side is).

## 2 Storage: pages are the cost

cost = pages read + written one I/O = $10^4$–$10^6$ instructions; CPU ignored

seq 16 $\mu$s/p, SSD rnd 100 $\mu$s, HDD rnd 10 ms 500 MB/s; `orders` 51 ms / 0.3 s / 32 s; `events` 16 s / 100 s / 2.8 h

record = columns + 24 header, rounded up to 8 PostgreSQL tuple

$r = \lfloor 8192 - 24 \rfloor / (\text{record} + 4\ \text{slot})$, floored rows per page

$B = \lceil N/r \rceil$, size = $8B$ KB `tickets`: 40+24=64, +4 → 120 rows/p, 25,000 p, 200 MB

**Slotted page**: header, slot directory (offset, length) growing down, tuples growing up; row id = (page, slot) = `ctid`; tuples move inside a page without touching indexes; big values out of line (TOAST).

| | heap | sorted | hashed |
|---|---|---|---|
| scan | $B$ | $B$ | $B$ |
| = present | $B/2$ | $\log_2 B$ | $\sim 1$ |
| = absent | $B$ | $\log_2 B$ | $\sim 1$ |
| range | $B$ | $\log_2 B$+pages | $B$ |
| insert | 1 | up to $B$ | 1 |

Real systems: heap + indexes. **VACUUM**: frees dead tuples inside pages, truncates only empty trailing pages (random delete of half: still 3,185 p). **VACUUM FULL**/`CLUSTER`: rewrite (1,592 p). **CPU-bound, not I/O**: hits high, reads ≈ 0, no speed-up warm (regex filter, jsonb, sort of cached data).

! Forgetting the slot (+4) or the floor. "Absent key costs $B/2$" (it is $B$). "VACUUM shrinks the file". "Column stores are always better".

## 3 Indexes

#leaves = $\lceil N/L \rceil$, $h = \lceil \log_F N \rceil$ bracket $N$ between $F^{h-1}$ and $F^h$; 2M at 200: 3; 20M at 250: 4

lookup = $h + 1$ +1 = the heap page; absent key: $h$

range, $k$ matches = $h + \lceil k/L \rceil - 1 + \text{heap}(k)$ heap($k$): $k$ unclustered, $\lceil k/r \rceil$ clustered, 0 index-only

$t = \sum$ reads × device 5 reads: 0.5 ms SSD, 50 ms HDD; top 2 levels resident: 0.3 ms

**Composite** `(a,b)`: sorted by `a`, then `b` within `a`: only a *leading prefix* descends; equality columns first, range column last; `b` alone, `a+0=7`, `ORDER BY b`: no. **Expression** `(lower(email))`; **partial** `WHERE event_type='purchase'`; **covering** `INCLUDE (amount)` → index-only scan; **BRIN** for append-ordered tables; **hash** = equality only.

index scan ≈ $h$ + leaves + 4 $sN$ one random heap read per match

pages touched = $B\,(1 - (1-s)^r)$ $sr \geq 3 \Rightarrow$ every page, in random order

plain index wins iff $4sN < B \iff s < 1/(4r)$ 0.16% on `orders`, 0.5% on `events`

**Bitmap scan**: row ids sorted by page, each page once in order: crossover a few %. **Correlation** ≈ ±1 (append-only, after CLUSTER): $s$ of rows = $s$ of pages, cheap ranges, BRIN; ≈ 0: formula above. Mock C4: 8.5%, corr −0.0015, 13 matches/page, $(0.915)^{157} \approx 10^{-6}$: all 3,185 p ×4 vs 3,185: **no**. **Price of an index**: space (≈ 16 B/row), $n + 1$ random page writes per insert (2–4× slower), planner effort; never used when $s \gg 1/(4r)$ (`status`, 90% `'view'`); keep PK/FK/join columns, one composite over many singles, drop `idx_scan = 0`.

! $\log_2 N$ for the height (it is $\log_F N$: 3–4, never 21). Counting rows, not pages. "I created it, so it is used". $(a, b)$ serving $b$ alone.

## 4 Sorting and joins

runs = $\lceil B/M \rceil$, fan-in = $M - 1$ pass 0 writes runs; one buffer for output

merge passes = $\lceil \log_{M-1} \text{runs} \rceil$ divide runs by $M - 1$ (ceil) until 1, count

I/O = $2B$ (1 + merge passes) read + write every pass

2 passes iff $\lceil B/M \rceil \leq M - 1$ ($M \approx \sqrt{B}$); 1 pass iff $M \geq B$ 800/21: 39 runs, 3 passes, 4,800; 6,000/100: 2 passes, 24,000; 1,200/25→50: 7,200→4,800

**PostgreSQL**: `Sort Method: quicksort Memory` fits; `external merge Disk` spilled; `work_mem` is per sort/hash per connection: `SET LOCAL`.

| | |
|---|---|
| simple NL | $B_R + \lvert R \rvert\, B_S$ (rows!) |
| block NL | $B_R + \lceil B_R/(M-2) \rceil B_S$ |
| outer fits, $B_R \leq M - 2$ | $B_R + B_S$ |
| index NL | $B_R + \lvert R \rvert (h + m)$, $m = \lvert S \rvert / d_S$ |
| sort-merge | $\text{sort}(R) + \text{sort}(S) + B_R + B_S$ |
| hash, build fits | $B_R + B_S$ (build = smaller) |
| Grace hash | $3(B_R + B_S)$ (`Batches`> 1) |

Mock 1.5: 300, 3,000, $M = 32$: $\lceil 300/30 \rceil = 10$: 30,300 (traps 9,300 / 900,300 / 3,300). B3: 80, 3,200, $M = 10$: NL 32,080, Grace 9,840; $M = 100$: both 3,280 (tie). 3.4b: users 30,000 p > 8,000: Grace 3,090,000 vs sort-merge 5,150,000 vs BNL 4,030,000. **Best / disaster**: BNL: one input fits, any condition / two big inputs. Index NL: tiny outer + index (`c.id=42`: 1 + (3 + 1) p vs 3,185) / big outer (`loops=20000`, 641 ms vs 232). Sort-merge: inputs sorted or sorted output wanted, ranges / nothing sorted, little memory. Hash: big equi-join, build fits / non-equality, build ≫ $M$, skew. `enable_*` are $10^{10}$ penalties, not bans. **Aggregation**: HashAggregate (few groups, breaker) vs Sort + GroupAggregate (many groups or sorted input, streams).

! $|R|$ where $B_R$ belongs. Fan-in $M$ (it is $M - 1$); forgetting pass 0; counting reads only. "Doubling memory halves the sort" (removes at most one pass). "Hash join for everything".

## 5 Optimisation: estimates and costs

col = $v \in$ MCV : $s = \text{freq}(v)$ exact; `country='MA'`: 0.20

col = $v \notin$ MCV : $s = \frac{1 - \text{null_frac} - \sum \text{MCV}}{\text{n_distinct} - \#\text{MCV}}$ no MCV list: $1/d$; `cust_id=42`: 50 rows

range: $s$ = buckets below $c$ (+ linear share)/#buckets 100 equi-depth buckets; `amount>95`: 10%; BETWEEN: frac($b$)−frac($a$)

$s_{A \wedge B} = s_A s_B$, $s_{A \vee B} = s_A + s_B - s_A s_B$, $s_{\neg A} = 1 - s_A$ independence; correlated ⇒ product *under*-estimates

rows = $sN$, pages = $\lceil \text{rows}/r \rceil$

$|R \bowtie S| = |R||S|/\max(d_R, d_S)$ key–FK: = the FK side; $m = |S|/d_S$ matches per outer row

q-error = max(est/act, act/est) < 2 fine, 10 red flag; errors multiply upward: fix the *lowest* wrong node

**Cost units** (not ms): seq page 1, random page 4, row 0.01, index entry 0.005, operator 0.0025.

Seq Scan = $B + 0.01N + 0.0025\,qN$ `orders`: 8,185 no WHERE, 9,435 with one

Index scan (unclust.) = $4h + 4m + 0.015m$ per outer row / per lookup

hash join CPU $\approx 0.0025(|R|+|S|)+0.01|\text{out}|$

**Three failures and fixes**: stale statistics (`n_live_tup` ≠ `reltuples`) → `ANALYZE t`; correlated columns (`country AND city`: $0.2 \times 0.1$ vs true 0.1) → `CREATE STATISTICS s (dependencies) ON a,b FROM t; ANALYZE t`; skew (`event_type` 90/9.5/0.5) → `ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000`. Mock 3.1: leaf `rows=40` vs 200,000 (5,000×) → nested loop re-scanning 1M pages per outer row; after the fix: hash join, ≈ 1,030,000 p. **Join ordering**: $n$ tables: $n!$ left-deep orders (5: 120), System-R DP memoises $2^n - 1$ subsets (31); bushy more. Estimation is the hard part: every later choice is a function of row counts. **Levers** (no hints): statistics, `random_page_cost` (1.1 on SSD), `work_mem`, `enable_*` for experiments, `pg_hint_plan` last.

! Adding selectivities for AND (52,500 instead of 250). min or $d_R + d_S$ in the join size. Forgetting a cost term (3,185 alone). Treating the symptom (an index, `enable_hashjoin=off`) when `rows` is 1,000× off.

## 6 Buffer management

EAT = $h\,T_{\text{hit}} + (1-h)\,T_{\text{miss}}$ 99% on HDD still ≈ 100 $\mu$s; 98% on SSD 2.1 $\mu$s

$h$ = hit/(hit + read) `Buffers: shared hit=40 read=3145`: 1.3%; second run 100%

`read` = not in `shared_buffers` (OS cache or disk; cold/warm gap only 1.3×); `temp read/written` = a sort or hash spilling, not the pool. **Policies**: OPT (future), FIFO (no renew on hit, Belady's anomaly), LRU (renew on hit; 0% on a scan larger than memory), **clock-sweep** = LRU approximation with `usage_count` 0–5, hand clears and evicts the first 0. **Sequential flooding**: one big scan evicts every hot page; PostgreSQL: a scan of a table > `shared_buffers`/4 uses a 256 KB (32-page) **ring**; COPY/VACUUM rings too. Cold 1M-page scan through 500,000 frames: $h \approx 0$. **Steal** (dirty pages of uncommitted work may be written) needs *undo*; **no-force** (commit does not write data pages) needs *redo*; real systems: steal + no-force + **WAL**: log record to disk *before* its data page, log forced at commit (one sequential append, 2 ms for 300 pages); crash: redo committed, undo uncommitted. **Sizing**: `shared_buffers` ≈ 25% of RAM; 80% starves the OS cache, `work_mem`, checkpoints; measure hit ratio and `pg_buffercache` first.

! "read = disk". "LRU is always better than FIFO" (not on a scan). "Commit writes the pages" (it writes the log). Pinning pages that are in use is not optional.

## 7 Reading a plan

`(cost=start..total rows=est width=)` = the planner's guess; `(actual time=first..last rows=act loops=n)` = measured, *per loop*; total = rows × loops. `Filter` + `Rows Removed`: predicate tested per row; `Index Cond`: answered by the index; `Recheck Cond` + `Heap Blocks: exact=`: bitmap scan, pages touched; `Heap Fetches: 0`: index-only; `Hash Cond`/`Merge Cond`; `Buckets Batches Memory Usage`; `Sort Key`, `Sort Method`; `Group Key`. **Method**: leaves → walk up → est vs act from the leaves (q-error) → Buffers/Sort Method/Batches/loops → decide + fix. **EXPLAIN q**: plan + estimates, does not run. **EXPLAIN ANALYZE**: runs, adds actuals, Sort Method, Batches, Execution Time (wrap DML in BEGIN/ROLLBACK). **EXPLAIN (ANALYZE, BUFFERS)**: + hit/read/temp per node.

## 8 Instruments and SQL to type

```
SELECT reltuples::bigint, relpages
  FROM pg_class WHERE relname='t';
SELECT pg_relation_size('t')/8192;
  pg_total_relation_size('t');
  pg_indexes_size('t');
  pg_size_pretty(...)
SELECT count(*) FROM
  heap_page_items(
  get_raw_page('t', 0)); --
  pageinspect; ctid = (page, slot)
SELECT attname, n_distinct,
  most_common_vals,
  most_common_freqs,
  histogram_bounds, correlation
  FROM pg_stats WHERE
  tablename='t'; -- n_distinct<0:
  fraction of rows
SELECT n_live_tup, last_analyze
  FROM pg_stat_user_tables WHERE
  relname='t';
ANALYZE t; VACUUM ANALYZE t;
CREATE STATISTICS s_geo
  (dependencies) ON country, city
  FROM customers; ANALYZE
  customers;
ALTER TABLE t ALTER COLUMN c SET
  STATISTICS 1000; ANALYZE t;
SET work_mem='64kB'; SET
  enable_hashjoin=off; RESET ALL;
  SHOW shared_buffers; BEGIN; SET
  LOCAL work_mem='64MB'; ...;
  COMMIT;
CREATE EXTENSION pg_buffercache;
  SELECT c.relname, count(*) FROM
  pg_buffercache b JOIN pg_class c
  ON b.relfilenode =
  pg_relation_filenode(c.oid)
  GROUP BY 1 ORDER BY 2 DESC;
SELECT indexrelname, idx_scan FROM
  pg_stat_user_indexes WHERE
  relname='t'; -- idx_scan = 0:
  drop
COPY (SELECT ts FROM events) TO
  '/d/ts.parquet' (FORMAT
  parquet); -- DuckDB: SELECT
  ts::date, count(*) FROM
  read_parquet('...') GROUP BY 1;
EXPLAIN q; EXPLAIN ANALYZE q;
  EXPLAIN (ANALYZE, BUFFERS) q;
  \timing on; \d t; \x
```

**Indexes**

```
CREATE [UNIQUE] INDEX ON t (a, b);
  ON t (lower(email)); ON t
  (user_id) WHERE
  event_type='purchase';
CREATE INDEX ON t (a) INCLUDE (b);
  ON t USING brin (ts); ON t USING
  hash (email); DROP INDEX name;
```

**Report (Mock 1 3.7a)**

```
SELECT u.country, count(*) AS n
  FROM events e JOIN users u ON
  u.user_id = e.user_id WHERE
  e.event_type='purchase' AND e.ts
  >= now() - interval '30 days'
  GROUP BY u.country ORDER BY n
  DESC;
```

**Skewed data (3.7b): one draw, several cuts, then ANALYZE**

```
INSERT INTO events (...) SELECT
  ..., CASE WHEN r<0.90 THEN
  'view' WHEN r<0.995 THEN 'cart'
  ELSE 'purchase' END, ... FROM
  (SELECT random() AS r FROM
  generate_series(1,1000000)) g;
  ANALYZE events;
```

! `random()` inside each WHEN (two draws). Alias in WHERE. `EXPLAIN ANALYZE DELETE` outside a transaction. `CREATE STATISTICS` without the `ANALYZE` after it.

## 9 Tickets, one line each (claim, mechanism

**"shared_buffers = 80% of RAM"**: no; PostgreSQL relies on the OS cache for reads (a `read` is often a cache hit), `work_mem` and the checkpointer need the rest; 25%, then measure hit ratio and `pg_buffercache`. **"Hash join is always best, disable the others"**: nested loop wins for a tiny indexed outer ($1 + (h+1)$ p vs 3,185) and is the only non-equi join; merge join wins on sorted inputs/sorted output; `enable_*` are penalties; every point query becomes a table scan. **"Commit takes 2 ms but changed 300 pages: how is it safe?"**: commit forces only the log (sequential append); data pages stay dirty (no-force); after a crash redo replays committed records, undo (steal) reverses uncommitted ones. **"Index every column"**: an index is used only below $s \approx 1/(4r)$; each costs space, $n+1$ writes per insert, planner time; keep PK/FK/join/filter columns, drop `idx_scan = 0`. **"Why count pages, not instructions"**: one I/O = $10^4$–$10^6$ instructions, hardware-independent, computable from $B, N, M$; wrong only when CPU-bound on cached data (hits high, no warm speed-up). **"Why an own buffer pool"**: access pattern (rings vs OS LRU flooding), write ordering (WAL before data), pinning and memory budget (double buffering). **"Estimation is the hardest part"**: every later decision (algorithm, order, memory) is a function of row counts; errors at a leaf multiply through joins; search (DP) is solved, statistics are not. **"Doubling memory halves the sort"**: passes = $1 + \lceil \log_{M-1} \text{runs} \rceil$; doubling removes at most one pass; single pass only at $M \geq B$. **"VACUUM will shrink the table"**: only inside pages and trailing empty pages; `VACUUM FULL/CLUSTER` rewrite. **Answer shapes**: quick check = letter + one number or formula; situation = data, formula, computation with units, one-sentence decision; plan = node, est vs act, Buffers line, fix; ticket = claim, mechanism, number, consequence.